

DGL-26 Kaggle: *Adjacency Dwags, 6th Place*

Amit Chandra, 06063187, ac4625 — Igor Lima Rocha Azevedo, 02487347, il225,
Iris Liao, 06071562, idl25 — Mikolas Fromm, 06055049, mf1025

I. METHODOLOGY & NOVELTY (40 POINTS)

A. Problem description & motivation (5 points)

The graph super-resolution problem asks how to accurately predict edge weights (and possibly node and edge feature embeddings) of a high-resolution graph using only a low-resolution graph as input. As discussed by Professor Rekik in Lecture 1, this can be applied to areas such as MRI brain scans for patients in poorer or more remote regions where equipment can only produce LR graphs. These LR graphs must be upsampled before being used by existing processes trained on HR data (perhaps by richer western countries).

The literature identifies key challenges. First, unlike simple image upsampling, the non-Euclidean nature of these networks means that relationships between points may be too complex for traditional methods. For example, Isallari et al., 2021 find that HR graphs created using GAN methods typical of image upscaling are susceptible to variability and bias due to required preprocessing steps [1]. Second, GNNs performing node feature learning do not attempt to preserve topology when shifting to HR. This means that key information needed for applications such as identifying diseases like Alzheimer’s and Parkinson’s gets lost [2]. Finally, for problems such as mapping brain topology, the dense interconnections cause computational costs to skyrocket [1] [2].

Additionally, a practical challenge is that while lectures have taught us standard methods for equal-node graph generation (e.g. Graph U-Net and Graph VAE), we have not learned methods that intelligently increase the number of nodes compared to input data, as is required for graph super-resolution.

B. State-of-the art methods (3 points)

Table I summarizes various SOTA methods.

C. Main figure (4 points)

The proposed *RefineU* architecture is illustrated in Fig. 3. The graph pooling and unpooling operations are adapted from the Graph U-Net framework introduced by Gao and Ji [5], while the brain graph super-resolution paradigm follows the formulations of Isallari and Rekik [6] and Singh and Rekik [2].

D. Brief overview of the proposed GNN (5 points)

The proposed model starts with a standard graph U-Net architecture consisting of multiple encoder-decoder GCN layers, similar to the design of GSR-Net [6]. It takes only the adjacency matrix of an LR graph as input. It follows the same logic of graph pooling and unpooling along the U path, but replaces the traditional skip connection with a learnable *skip*

TABLE I: Overview of GNN-based SOTA Models

Model	Brief Description
AGSR-Net [1]	One of the first GNN frameworks to predict HR graphs from LR input. Generates HR graphs by adding a super-resolution decoder on top of a standard two-encoder / two-decoder U-Net. Uses hybrid loss accounting for both self-reconstruction and super-resolution (as well as eigen decomposition). Regularises through an adversarial model detecting whether an HR graph is synthetic or observed.
STP-GSR [2]	The first super-resolution method to learn in a higher-order space to better preserve graph topology for disease prediction. Initialises an HR graph from LR input using matrix multiplication. Then translates HR edges into dual graph nodes to allow for simple node updates. Finally uses updated dual graph nodes to update HR graph’s edge weights. Computationally efficient due to high level of sparsity of dual graph.
Bi-SR and DEFEND [3]	Alters the dual graph system found in STP-GSR by first translating LR graphs into initial HR by using a fully-connected bipartite LR-HR graph to update HR nodes (Bi-SR). Then proceeds with using a dual graph as in STP-GSR to infer edge weights (DEFEND).
BGSR [4]	Uses GANs to generate HR brain graphs from LR input directly. Clusters training data based on topological features such as degree centrality. Samples the nearest neighbors in the training data to the test LR input to predict HR graphs.

gate. This allows it to modulate the amount of information flowing from lower U-Net layers.

Apart from altering the skip connection mechanism, the proposed model differs from GSR-Net by replacing the final GSR layer after the graph U-Net pass [6] with a Feedforward Neural Network (FNN) that directly predicts the adjacency matrix of the HR graph. This approach equips the FNN with a global receptive field such that each HR graph node is calculated using a feature representation of all LR graph nodes.

The model is trained using a combination of two weighted losses – the difference between predicted and target adjacency, and the difference between the input and output of the UNet.

Finally, the proposed model also differs by the node feature initialization. As discussed in Lecture 6, additional topological features have been calculated and appended to the node’s original identity feature matrix. These can help the decoder understand the role of nodes and their neighbours in the larger structure during reconstruction. The two features added are betweenness centrality ($\sum \sigma_{st}(v)/\sigma_{st}$) and spectral encoding.

Of the topological features added, the most complex is spectral encoding. To address the absence of intrinsic node features in the provided connectivity matrices, we inject structural inductive biases to provide the model with immediate topological context. One of the initializations we use is spectral encoding in order to establish a global coordinate system

for brain regions. Given the low-resolution adjacency matrix $A^{LR} \in \mathbb{R}^{160 \times 160}$, we compute the symmetric normalized graph Laplacian as $L = I - D^{-1/2} A^{LR} D^{-1/2}$ where I is the identity matrix and D is the diagonal degree matrix. By solving the eigenvalue problem $L\mathbf{u} = \lambda\mathbf{u}$ we extract the top-16 eigenvectors corresponding to the smallest non-zero eigenvalues. These eigenvectors form a spectral feature matrix $U \in \mathbb{R}^{160 \times 16}$, where the i -th row serves as a 16-dimensional positional encoding for node i . Intuitively, because the graph Laplacian measures signal smoothness across the network, tightly connected nodes will possess similar values in these lower-frequency eigenvectors. This approach endows nodes with macroscopic information, with the idea being that they can encode the functional communities of the brain into a Euclidean space, giving the model awareness of the global context before local message-passing.

E. Innovative components (10 points)

This report proposes three novel components, highlighted in Table II. The network is trained on the following loss,

$$\mathcal{L} = \mathcal{L}_{hr} + \lambda \mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \left(\frac{1}{2} \|\mathbf{A}_{hi}^{pred} - \mathbf{A}_{hi}^{target}\|_2^2 + \frac{1}{2} \|\mathbf{A}_{hi}^{pred} - \mathbf{A}_{hi}^{target}\|_1 \right) + \lambda \frac{1}{N} \sum_{i=1}^N \|\mathbf{z}_{in}^{unet} - \mathbf{z}_{out}^{unet}\|_2^2$$

The objective function combines $L1$ and $L2$ losses to minimize the Mean Absolute Error (MAE) and Mean Squared Error (MSE) of the final predictions. Additionally, an $L2$ reconstruction loss is employed to minimize the MSE between the latent UNet input and output representations. Based on the ablation studies illustrated in Figure 1, the scaling factor λ was set to either 0.0 or 0.2. To avoid an exhaustive grid search, we evaluated only these two configurations; Table V specifies which value yielded the best performance for each model.

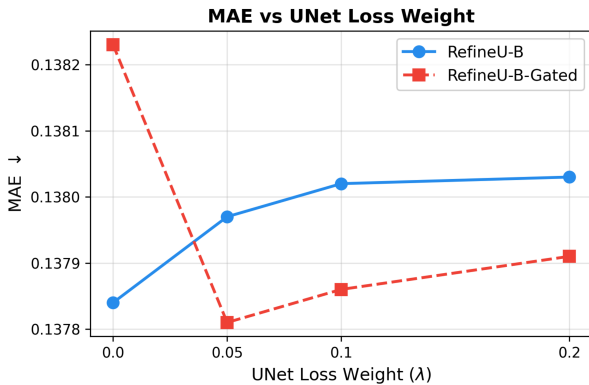


Fig. 1: Effect of UNet auxiliary loss weight (λ) on MAE for two different model configurations for *RefineU*. RefineU-B (betweenness and non-gated) degrades monotonically as λ increases, while RefineU-B-Gated (betweenness and gated) benefits from a small auxiliary loss, achieving its best MAE at $\lambda=0.05$.

TABLE II: Innovative Components of the Proposed GNN Framework

Contribution	Rationale
Initialised node features with topological metrics in addition to identity.	GSR-NET and AGSR-NET only use identity for initial node features. These collect edge weight information during embedding updates [6] [1]. However, as mentioned in Lecture 6, an alternative is to initialise with topological features, which may provide data of the macroscopic structure. Of these, spectral encoding is the most innovative and is discussed in depth in section I.D
Skip connections replaced with skip gates.	While plain skip connections treat all encoder features with equal weight, this approach can inadvertently introduce noise or task-irrelevant patterns into the decoder. By contrast, a gated mechanism learns to assign per-feature importance by calculating a sigmoid-activated weight based on the local context. This allows the network to dynamically suppress unhelpful information and amplify critical features at each level of the hierarchy. Consequently, the gated skip connection provides a more expressive, task-specific mapping than the rigid addition used in standard architectures.
Replaced the GSR layer of GSR-NET with an FNN	Replacing the GSR layer with a FNN leverages the fixed canonical ordering of brain graphs to learn localised, region-specific mapping rules that traditional GCNs overlook. While FNNs are permutation-variant, the UNet backbone ensures initial neighborhood-aware feature aggregation, providing a structured representation for the FNN to generate high-resolution adjacency matrices. Additionally GSR upsamples a graph signal while aiming to retain the frequency domain characteristics of the original signal. This assumes that HR brain networks are simply "smoother" or higher-frequency extensions of the LR topology, a strong inductive bias that may not hold. The MLP does not enforce this same bias. The spectral GSR layer also has a high computational cost because it requires the explicit computation of the eigenvector matrix U_1 of the target HR graph Laplacian during training. It then utilizes an eigen-decomposition loss ($\mathcal{L}_{eig}$) to minimize the mean squared error between the learned weights and these target eigenvectors. Calculating the eigen-decomposition of $N \times N$ matrices at every forward pass is computationally expensive and potentially numerically unstable for large batches.

TABLE III: Additional Topological/Geometric Measures

Measure Name	Brief Description & Rationale
Closeness Centrality MAE (or MSE)	Closeness centrality denotes the average shortest distance between a node and any other node [3]. For brain topology where the graph is weighted, this error metric will measure how well the generated graph depicts major brain pathways compared to ground-truth.
Degree Centrality MAE (or MSE)	Degree centrality depicts the average number and strength of connections for each node. This error metric will measure how closely the generated graph matches the overall connectivity levels found in the ground-truth data.

F. Mathematical properties of the proposed GNN (13 points)

Permutation invariance (5 points)

- a) Permutation invariance in mathematics is when $f(x) = f(t(x))$ for function $f()$ and transformation $t()$, meaning x is invariant to $t()$. For GNNs, a function f_k is invariant if for any permutation matrix P for each layer of a GCN $f(A, H_k) = f(P^T A P, H_k P)$. (1 point)
- b) Let the output of the U-Net encoder be the node embedding matrix $H \in \mathbb{R}^{N_{lr} \times d}$, where $N_{lr} = 160$. The MLP head flattens this matrix into a single vector $\mathbf{h} = \text{vec}(H) \in \mathbb{R}^{N_{lr}d}$. If we apply a permutation P to the input graph, the equivariant encoder yields a permuted embedding matrix PH . Vectorizing this permuted output results in:

$$\text{vec}(PH) = (I_d \otimes P)\text{vec}(H) = (I_d \otimes P)\mathbf{h}$$

The first layer of the MLP applies a learned weight matrix W and bias $\mathbf{b}$, producing $W(I_d \otimes P)\mathbf{h} + \mathbf{b}$. For the model to be permutation invariant, this output must perfectly equal the non-permuted output $W\mathbf{h} + \mathbf{b}$ for all possible permutations P and hidden states $\mathbf{h}$. This would require $W(I_d \otimes P) = W$, meaning the weights in W would have to be perfectly uniform across all node dimensions, effectively ignoring the spatial identities of the nodes. This equality quality holds only for a restricted set of weight matrices. Since the architecture does not enforce any constraints, a generic MLP head will not satisfy permutation invariance. It should be noted that MLPs can “learn” a weak form of permutation invariance from data. (4 points)

Expressiveness (3 points)

- a) Expressiveness is the ability of a graph-based model or function to provide different outputs for either different topologies or different values. For example, max pooling is not expressive when it treats a node with two neighbouring nodes of value 2 the same as a node with ten neighbours of value 2. It is also not expressive when it treats a node with two neighbours of value 2 the same as a node with one neighbour of value 1 and one neighbour of value 2. (1 point)
- b) Our proposed model **RefineU** is highly expressive. Our GCN utilizes mean aggregation, which would normally fail to distinguish between different-sized neighbourhoods of identical values. However we have initialized nodes with continuous topological features (betweenness centrality and Laplacian eigenvectors). We can thus make rich representations such that nodes of different topology very likely have different features. At the readout stage, the MLP flattens these node embeddings into a single vector, ensuring that any structural variation in the input graph produces a different high-resolution output matrix. (2 points)

Permutation equivariance (5 points)

- a) Permutation equivariance in mathematics is when $f(t(x)) = t(f(x))$ for function f and transformation t , meaning f and t are equivariant. For GNNs, node permutation equivariance is a special case where for a permutation matrix P for every layer of a GCN $H_{k+1}P = F(H_k P, P^T A P, \phi_k)$ (1 point)
- b) Let $X \in \mathbb{R}^{N_{lr} \times d}$ denote the node embedding matrix produced by the RefineU encoder, where $N_{lr} = 160$ and d represents the hidden feature dimension. The MLP head initiates the graph-to-graph translation by flattening this embedding matrix into a 1D vector via the vectorization operator, such that $\mathbf{h} = \text{vec}(X) \in \mathbb{R}^{N_{lr}d}$. Consider an arbitrary node permutation matrix $P \in \{0, 1\}^{N_{lr} \times N_{lr}}$. Applying this permutation to the input graph results in a permuted embedding matrix PX . The vectorization of this permuted matrix can be expressed using the Kronecker product $\otimes$ as:

$$\text{vec}(PX) = (I_d \otimes P)\text{vec}(X) = (I_d \otimes P)\mathbf{h}$$

where I_d is the identity matrix of size d . The subsequent dense linear layer applies a learned weight matrix $W \in \mathbb{R}^{m \times N_{lr}d}$ and a bias vector $\mathbf{b}$, yielding the output:

$$f_{MLP}((I_d \otimes P)\mathbf{h}) = W(I_d \otimes P)\mathbf{h} + \mathbf{b}$$

For this layer to satisfy permutation equivariance (or structural invariance mapping to a fixed output space), there must exist a predictable permutation operator P_{out} such that the output corresponds to $P_{out}(W\mathbf{h} + \mathbf{b})$ for all $\mathbf{h}$ and P . This strict condition necessitates that $W(I_d \otimes P) = P_{out}W$. Because W is a dense, unconstrained matrix optimized during training, this commutativity equality does not hold for arbitrary permutations. Vectorization irreversibly entangles the spatial node dimension with the hidden feature dimension, assigning specific learned weights in W to specific node indices. Thus, the architecture strictly breaks permutation equivariance.

While the lack of permutation invariance or equivariance is usually viewed as a limitation in general graph learning problems, in our specific case, the nodes in the connectivity matrices $A^{LR} \in \mathbb{R}^{160 \times 160}$ and $A^{HR} \in \mathbb{R}^{268 \times 268}$ are not anonymous, interchangeable entities. Instead, they are canonically ordered, representing fixed regions in the brain. (4 points)

II. EXPERIMENTAL SETUP & EVALUATION (27 POINTS)

A. Results (9 points)

- a) See Table III for a detailed explanation on each topological feature introduced. (2 points)
- b) (4 points) Figure 2 presents the findings from our 3-fold cross-validation on the initial dataset of 167 samples across all 8 evaluation measures. As detailed in Table

V, RefineU demonstrates strong generalization, achieving the best or near-best performance on 5 of the 8 evaluated metrics. While other baseline approaches slightly lead on raw MAE and PCC, RefineU exhibits dominant performance on topology-aware metrics (such as CC, DC, and BC). This demonstrates that our model produces high-resolution graphs that are not only edge-accurate but also structurally faithful, which is a critical property for downstream brain connectivity analysis. Architecturally, replacing the standard *GSRLayer* with a more general, permutation-variant *FNN* allows the model to learn highly specific and localised mapping rules for exact brain regions, given their fixed canonical ordering. The UNet backbone aggregates low-resolution features with respect to their local neighborhood, creating a rich representation upon which the *FNN* generates the high-resolution adjacency matrix. To mitigate the risk of overfitting, we include a dropout ($p = 0.2$) regularization.

We have also experimented with the final output activation, using $\text{Identity}(A_{hi}^{\text{predicted}})$, $\text{ReLU}(A_{hi}^{\text{predicted}})$, $\text{LeakyRelu}(A_{hi}^{\text{predicted}})$ and $|A_{hi}^{\text{predicted}}|$. We ended up using *LeakyRelu* for its ability to generate 0.0-weighted edges the best, while not affecting the low-weighted edges distribution too drastically. We evaluate all model variants ablating four design axes: (i) node features — Identity ($N \times N$ learned), Eigen (top-16 Laplacian eigenvectors), Betweenness centrality, or All (concatenation); (ii) gated skip connections — a learned sigmoid gate controlling encoder-to-decoder information flow vs. direct additive skips; (iii) output activation — LeakyReLU vs. None on the predicted HR adjacency; and (iv) UNet auxiliary loss weight λ_{UNet} — an MSE loss between encoder-entry and decoder-exit embeddings, set to 0.0 (disabled) or 0.2.

- c) (2 points) The full 3-fold CV (3 models $\times$ 50 epochs) trains in 2.2 minutes (130 seconds) on an NVIDIA GeForce RTX 3060 Ti. Peak RAM usage is 1.8 GB and peak GPU memory is 397 MB. The model has 19.9M parameters, but training remains fast because graph operations act on small adjacency matrices (160 $\times$ 160) that further shrink through TopK pooling (ratio 0.7 per level), and the sample-by-sample training loop (batch size 1) keeps memory consumption low.
- d) (1 point) For the final submission, we use ensemble prediction: all 3 fold models trained during CV are retained, and each independently predicts on the 112 test samples (test_LR). The final test_HR prediction is the element-wise average of the 3 fold predictions, clipped to [0, 1]. This ensemble approach avoids the need for a separate full-dataset retrain and reduces variance by averaging out fold-specific biases. Our final Kaggle score is 0.148417, ranked 6th.

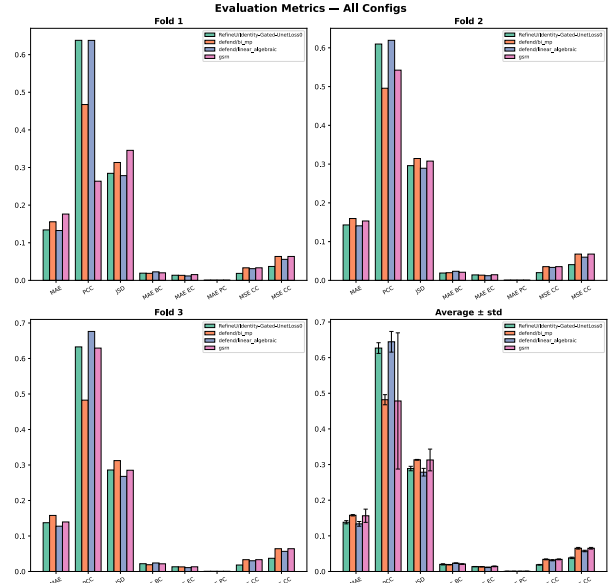


Fig. 2: Performance comparison of RefineU and baseline approaches across eight evaluation measures using 3-fold cross-validation

B. Comparison Against Other Methods (6 points)

The Table V shows a in depth analysis made across different models and configuration. Our best **RefineU** configuration (Identity features, LeakyReLU, no gating, $\lambda_{\text{UNet}} = 0.0$) achieves a CV MAE of 0.13750 ± 0.00373 and MSE of 0.03049 ± 0.00193 , the lowest MSE across all models tested including baselines. Our second-best variant adds gated skip connections (Identity features, LeakyReLU, gated, $\lambda_{\text{UNet}} = 0.0$), achieving MAE 0.13768 ± 0.00367 and MSE 0.03056 ± 0.00188 — nearly matching the best while exhibiting the lowest MSE standard deviation among all **RefineU** models, suggesting that gating slightly improves stability across folds at a marginal cost in average performance. Compared to baselines, **RefineU** outperforms GSRN (MAE 0.15951), DEFEND-BiMP (0.15883), DEFEND-BiLC (0.15172), and all GCN variants on MSE. DEFEND-LA achieves the best MAE (0.13549) but at a notably worse MSE (0.03158 vs 0.03049), indicating that **RefineU** produces tighter predictions overall despite slightly higher average absolute error.

C. Scalability of Your Proposed GNN Model (7 points)

- a) **RefineU** (UNet GCN component) Bottleneck: The model utilizes dense adjacency matrices, imposing an $O(N^2)$ spatial complexity and an $O(N^2d)$ temporal complexity for graph convolutions (AX). While this allows for precise representation of all possible edge relationships in brain connectomes, it is not optimal for scaling to high-resolution graphs (where $N \gg 10^4$). In such cases, the memory overhead during the pooling and unpooling phases — where intermediate graph states must be stored for skip connections — becomes a limiting factor.

- b) Hierarchical efficiency: Despite the dense matrix costs, the U-Net architecture provides a scalable advantage over flat GCNs by progressively reducing the graph size. By contracting the node space in the encoder, the model performs its most computationally intensive feature extraction at lower resolutions, effectively mitigating the total FLOP count compared to a standard deep GCN operating at full resolution.
- c) FNN Parameter Explosion: The transition from the graph latent space to the target resolution represents a significant scaling bottleneck. In full-rank mode, mapping flattened features to an N_{hr}^2 target results in an $O(N_{lr} \cdot d_{hidden} \cdot N_{hr}^2)$ parameter explosion. The model’s scalability is strictly contingent on the low-rank approximation ($k > 0$), which reduces the parameter footprint to $O(N_{lr} \cdot N_{hr} \cdot k)$. This allows the model to predict high-resolution outputs by learning a latent basis rather than an exhaustive edge-to-edge mapping.

D. Reproducibility of Your Proposed GNN Model (5 points)

We assessed reproducibility using 3-fold cross-validation (seed 42), creating three distinct train/test splits to vary the data distribution. As shown in Table V, consistency across folds is high; we observed a 3-fold standard deviation of ≤ 0.004 across all 16 model configurations. This stability—maintained regardless of feature type, gating, or activation choice—demonstrates that performance is not sensitive to the specific partitioning of training and validation samples.

III. DISCUSSION & REFLECTIONS (8 POINTS)

- a) Table IV shows in detail each of RefineU’s strengths and weaknesses. (4 points)
- b) (4 points) Surprisingly, none of the SOTA brain graph super-resolution methods use diffusion models for generation. Diffusion is SOTA for image tasks which may include graphs despite non-Euclidean geometry.

As noted in the literature, maintaining topological features is key to use the generated HR graph for downstream tasks like disease detection [1] [3]. Therefore, we propose using a diffusion model as the final step of generation / super-resolution after it has been informed by more traditional GCN and U-Net methods. In practice, this means replacing the FNN with a diffusion model, and making generation conditional on the node adjacency matrix and topological features passed from the final U-Net output. Loss will remain a combination of reconstruction and supervised error.

A second improvement might come from reframing the super-resolution question. In this competition the only data available are LR and HR adjacency matrices. However, it is possible that more information can be collected from even low-cost examinations of the brain, such as the condition of ROI nodes. This can allow the model to be trained to generate HR graphs with feature embeddings,

potentially allowing for better diagnosis. To enable this, the skip gate can pass input LR features to the reconstructed LR graph. The existing FNN can also predict initial HR node features. These can then be updated using GCN and the predicted HR edge weights.

TABLE IV: Strengths & Weaknesses of the Proposed GNN

Strength 1: Architecture based on simple, well-understood components.	Graph U-Net architecture has been around since 2019 [5], while feed-forward neural networks are the mainstay of most modern ML. The topological features are well known in graph and mathematics literature. Gated skip connections are a unique to this project, but the gating mechanism itself is used for many other applications [7].
Strength 2: Node features can be defined to allow any data for prediction.	While we have defined node features based on topology for this problem, the architecture allows for any relevant information to be considered for HR graph prediction. This information also gets refined through standard GCN node embedding updates.
Weakness 1: Our underlying method is from 2020 [6].	Our use of graph U-Net and initialisation of topological features is directly inspired from GSR-NET. One may argue that it is no longer SOTA. However, as our tests show, our modifications allow it to perform comparably to more recent paradigms.
Weakness 2: The model lacks compute scalability.	Our target size is $N_{hr} * N_{hr}$. The final linear layer requires $O(N_{lr} \cdot d_{hidden} \cdot N_{hr}^2)$ parameters. While this works for 268 nodes, this approach does not scale for massive graphs (e.g., 100,000+ nodes).

TABLE V: All trained models on 3-fold cross-validation (seed=42). For each **RefineU** variant, only the best-performing UNet loss weight is shown. **Bold**, underline, and dashed underline indicate the best, second-best, and third-best results, respectively. Between. stands for Betweenness and Act. for the output activation function.

Model	Features	U-Loss	Gated	Act.	MAE ↓	Std MAE	MSE ↓	Std MSE
RefineU	Identity	0.0	-	LReLU	<u>0.13750</u>	0.00373	0.03049	0.00193
	Between.	0.0	-	LReLU	0.13784	0.00397	0.03066	0.00206
	Eigen	0.0	-	LReLU	0.13822	0.00354	0.03083	0.00185
	All	0.2	-	LReLU	0.13829	0.00388	0.03090	0.00202
	Identity	0.0	✓	LReLU	0.13768	0.00367	<u>0.03056</u>	0.00188
	Between.	0.2	✓	LReLU	<u>0.13791</u>	<u>0.00386</u>	0.03069	<u>0.00198</u>
	Eigen	0.2	✓	LReLU	0.13836	0.00399	<u>0.03092</u>	0.00207
	All	0.2	✓	LReLU	0.13852	0.00389	0.03099	0.00202
	Identity	0.0	-	None	0.13806	0.00389	0.03069	0.00202
	Between.	0.0	-	None	0.13837	0.00410	0.03083	0.00211
	Eigen	0.2	-	None	0.13891	0.00410	0.03112	0.00212
	All	0.2	-	None	0.13877	0.00394	0.03104	0.00204
	Identity	0.0	✓	None	0.13836	0.00364	0.03082	0.00190
	Between.	0.2	✓	None	0.13832	0.00405	0.03081	0.00209
	Eigen	0.2	✓	None	0.13881	0.00399	0.03106	0.00208
	All	0.2	✓	None	0.13906	0.00404	0.03119	0.00211
GCN + MLP	Identity	-	-	-	0.13901	0.00391	0.03115	0.00204
	Between.	-	-	-	0.13958	0.00398	0.03144	0.00208
	Eigen	-	-	-	0.13845	0.00389	0.03088	0.00203
	All	-	-	-	0.13820	0.00372	0.03076	0.00194
DEFEND	Lin. Alg.	-	-	-	0.13549	0.00496	0.03158	0.00195
	BiLC	-	-	-	0.15172	<u>0.00353</u>	0.03474	0.00170
	BiMP	-	-	-	0.15883	0.00163	0.03953	0.00082
GSRN	-	-	-	-	0.15951	0.01509	0.04196	0.00707

Additional Resources: The full implementation is available on [GitHub](#) as well as on the submitted zip folder. A detailed step-by-step experiment log, including ablation curves and per-fold metrics, is also available as a [W&B Report](#).

REFERENCES

- [1] M. Isallari and I. Rekik, “Brain graph super-resolution using adversarial graph neural network with application to functional brain connectivity,” *Medical Image Analysis*, vol. 71, p. 102084, 2021.
- [2] P. Singh and I. Rekik, *Strongly Topology-Preserving GNNs for Brain Graph Super-Resolution*. Springer Nature Switzerland, Oct. 2024, p. 124–136. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-74561-4_11
- [3] —, “Rethinking graph super-resolution: Dual frameworks for topological fidelity,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.08853>
- [4] M. Roobini, M. S. Jebaraj, P. Sasirudh, K. Babu, and M. N. Muneera, “Brain graph super-resolution for enhanced neurological disorder diagnosis,” in *2024 International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems (ICSES)*, 2024, pp. 1–12.
- [5] H. Gao and S. Ji, “Graph u-nets,” 2019. [Online]. Available: <https://arxiv.org/abs/1905.05178>
- [6] M. Isallari and I. Rekik, “Gsr-net: Graph super-resolution network for predicting high-resolution from low-resolution functional brain connectomes,” 2020. [Online]. Available: <https://arxiv.org/abs/2009.11080>
- [7] A. V. Makkuva, S. Oh, S. Kannan, and P. Viswanath, “Learning in gated neural networks,” 2020. [Online]. Available: <https://arxiv.org/abs/1906.02777>

Refine-U Architecture

Graph UNet Encoder + MLP Decoder for Brain Graph Super-Resolution (160→268)

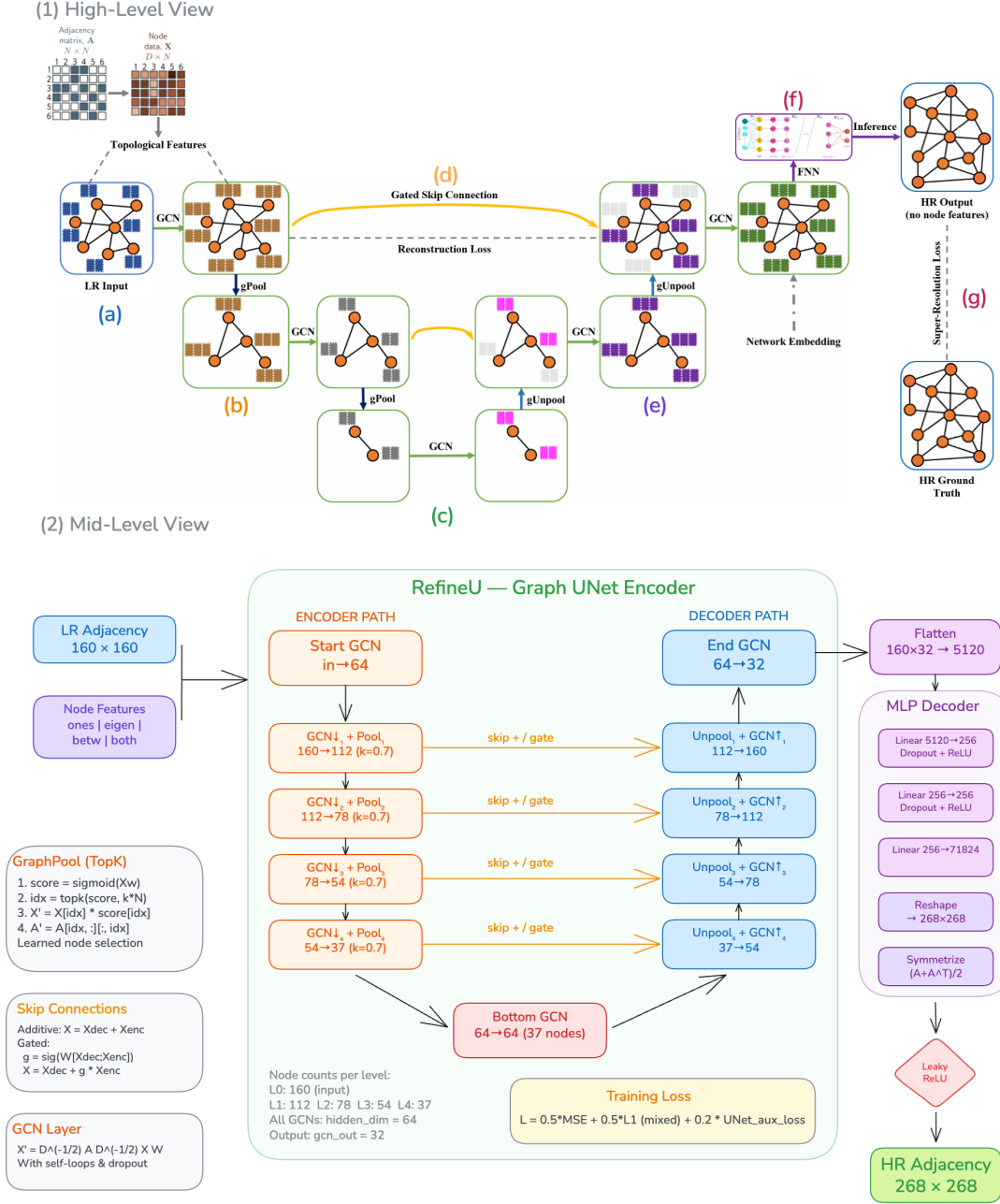


Fig. 3: Overview of the **RefineU** architecture for brain graph super-resolution ($LR : 160 \rightarrow HR : 268$ nodes). **(1) High-level view.** (a) The LR connectome is provided as an adjacency matrix $A \in \mathbb{R}^{N \times N}$ with topological node features (degree, betweenness centrality, eigenvectors) initialised as $X \in \mathbb{R}^{N \times D}$. (b) A hierarchical encoder applies alternating GCN layers and TopK graph pooling (gPool, $k=0.7$), coarsening the graph across four levels ($160 \rightarrow 112 \rightarrow 78 \rightarrow 54 \rightarrow 37$ nodes), with each encoder state cached for the decoder. (c) A bottleneck GCN operates at the coarsest resolution (37 nodes). (d) Learnable SkipGate modules connect each encoder level to its symmetric decoder counterpart via $X_{out} = X_{dec} + \sigma(W[X_{dec}; X_{enc}]) \odot X_{enc}$, suppressing noisy encoder features before fusion. (e) The decoder restores the original node count through four gUnpool + GCN stages ($37 \rightarrow 54 \rightarrow 78 \rightarrow 112 \rightarrow 160$). (f) The decoded embedding is flattened and passed through a three-layer MLP, projecting to a 268×268 matrix symmetrised via $(A + A^T)/2$ and activated with Leaky ReLU. (g) The predicted HR connectome is supervised via $\mathcal{L} = 0.5 \mathcal{L}_{MSE} + 0.5 \mathcal{L}_1 + 0.2 \mathcal{L}_{aux}$, where $\mathcal{L}_{aux}$ is a reconstruction loss between the first encoder and final decoder outputs which in some models it's not present. **(2) Mid-level view.** Explicit node counts, hidden dimensions, and data-flow annotations are shown for each sub-module alongside theoretical summaries of the GCN normalisation, TopK pooling, and skip-connection variants.